

Reviser-Extensible Grammars: A Q#-Targeted Quantum Front End for the Unknown-Prior Engine, Compiled and Interpreted in Bada

Abstract

Bada is a small object-oriented language whose runtime is the Unknown-Prior Engine — an append-only tuple ledger, a maximum-entropy prior, and an *entropy-phase* cognitive core that reweights a softmax distribution by a unit-modulus complex factor $\exp(i\theta_i)$ without ever destroying a confident zero. Bada already ships a C reference with a lexer, a recursive-descent parser, a tree-walking interpreter, and a Bada-to-C compiler, exposed through one executable, `bada`. This paper turns Bada's reflective `@reviser` construct — today a runtime learning loop — into a mechanism for **extending the language's own grammar at the level of committed facts**, and uses that mechanism to grow a *quantum* sublanguage modelled on Q#. The key observation is that the engine's native *phase* type, an amplitude $\psi_i = \sqrt{a_i} \cdot \exp(i\theta_i)$, is mathematically the same object as a quantum amplitude $\alpha_i = \sqrt{p_i} \cdot e^{i\phi_i}$. A reviser block that commits the production rules for `qubit`, `H`, `CNOT`, and `Measure` therefore realises a Q#-style front end directly on the engine's phase core, with no new numeric kernel: gates become phase commits, measurement becomes a ledger commit that sharpens a maximum-entropy prior, and the engine's two provable properties — zero-preservation and $|\psi|^2 = a$ — become, respectively, *no-resurrection of forbidden basis states* and *unitarity of the phase step*. We give the formal design of reviser-as-grammar-extension, the quantum sublanguage and its denotation on the phase core, a specialised quantum-inference library built from append-only measurement, and the compiler/interpreter changes that carry the extension from parse to native C. Each central claim is checked by a runnable probe on the existing executable: a Hadamard yields equal probabilities with $|\psi|^2 = a$ to 1.11×10^{-16} , and a forbidden transition (confident zero) survives a phase step exactly as a ruled-out token does.

1. Introduction

A language whose runtime is an inference engine invites an unusual question: can the language learn new *syntax* the way the engine learns new *facts*? Bada is a natural place to ask it. Bada was created to drive the Unknown-Prior Engine, and its design already carries a reflective construct, `@reviser`, whose stated role in the source manuscript is a learning loop that rewrites the engine's behaviour as evidence accumulates. In the current C reference, however, `@reviser` is modest: the parser skips its header up to the opening brace and the interpreter simply executes the block. It revises *values*, not *grammar*.

This paper makes `@reviser` revise grammar. The move is principled rather than opportunistic, because Bada's runtime is an *append-only ledger*: facts are never mutated, only committed, and the engine's whole epistemology is "start from maximum-entropy ignorance, then earn structure by appending evidence." Grammar extension fits that epistemology if a new production rule is itself a committed fact — a tuple in the ledger that the parser consults. A reviser block becomes a transaction that appends rules; the parser becomes a reader of the rule ledger; and syntactic growth becomes monotone and auditable in exactly the way the engine's inference already is.

We then put the mechanism to work on a demanding target: a *quantum* sublanguage modelled on Q#. The choice is not decorative. The engine's native *phase* type is a vector of unit-modulus complex numbers, and the cognitive core forms $\psi_i = \sqrt{a_i} \cdot \exp(i\theta_i)$ — literally an amplitude whose squared modulus is a probability. That is the defining shape of a quantum state-vector amplitude. A quantum front end is therefore the most honest stress test of reviser-extensible grammar, because the new syntax (`qubit`, `H`, `CNOT`, `Measure`) denotes operations the engine's core can already perform, and the engine's *theorems* turn into the quantum semantics one would want:

- **Zero-preservation** (a confident zero stays zero under any phase) becomes **no-resurrection**: a basis state with zero amplitude cannot be brought back by a gate, the quantum reading of a forbidden transition.
- $|\psi|^2 = a$ (the phase step changes no probability) becomes **unitarity of the phase layer**: gates that are pure phase are norm-preserving by construction.

1.1 Contributions

1. A formal redefinition (§3) of `@reviser` from a value-level loop to a **grammar-extension transaction** that commits production rules to the ledger, together with the parser-as-rule-reader discipline that makes the extension monotone,

scoped, and auditable.

2. A **Q#-style quantum sublanguage** (§4) — **qubit**, single-qubit gates, **CNOT**, and **Measure** — defined by reviser-committed rules and *denoted* on the engine's **phase** core, so that no new numeric kernel is introduced.
3. A **specialised quantum-inference library** (§5) in which measurement is an append-only commit that updates a maximum-entropy prior, giving a Bayesian, ledger-native account of repeated measurement and tomography.
4. **Compiler and interpreter realisation** (§6): how a reviser-extended grammar is carried from parse through the tree-walking interpreter and the Bada-to-C transpiler, reusing the existing native back end (**gcc**) unchanged.
5. **Runnable evidence** (§7) on the current executable: a Hadamard probe ($[0.5, 0.5]$, $|\psi|^2 = a$ to 1.11×10^{-16}) and a forbidden-transition probe (a zero amplitude preserved through a phase step), plus a candid account of the parser changes the full mechanism still requires.

We are explicit throughout about which claims run today on the reference and which are design proposals for the next revision of the parser.

2. Background

2.1 The Unknown-Prior Engine and Bada's phase core

The engine forms a base distribution $a = \text{softmax}(z)$ over V outcomes and then applies a cognitive step parameterised by per-module entropies H_m , phase fields ϕ_m , and gains β_m :

```
a_i = softmax(z)_i
theta_i = Sum_m beta_m * H_m * phi_m(i)
psi_i = sqrt(a_i) * exp( i * theta_i )
q_i = |psi_i|^2 / Sum_j |psi_j|^2
```

Because $|\exp(i\theta)| = 1$, the squared modulus is unchanged: $q_i = a_i$ exactly. Two properties follow and are verified at runtime by the reference: confident zeros are preserved ($a_i = 0 \Rightarrow q_i = 0$), and the maximum deviation of q from a is at the machine-epsilon scale. The native **phase** type is precisely the vector **psi**, a unit-modulus-factored amplitude.

2.2 Q# in one paragraph

Q# is a language for gate-model quantum computation. A register of qubits holds a state vector of complex amplitudes; gates are unitary matrices applied to that vector; **H** (Hadamard) creates uniform superposition; **CNOT** entangles; and **Measure** collapses the state to a basis outcome with probability equal to the squared amplitude, committing a classical bit. The amplitude of basis state i is $\alpha_i = \sqrt{p_i} * e^{i\phi_i}$. The correspondence with the engine is immediate: $p_i \leftrightarrow a_i$, $\phi_i \leftrightarrow \theta_i$, $\alpha_i \leftrightarrow \psi_i$. Measurement's probability rule is the engine's $|\psi_i|^2$, and measurement's classical record is a ledger commit.

2.3 The reviser today

In the reference, **@reviser** parses by skipping its header to the first brace and executes the enclosed block. It can rebind values and commit facts, but it cannot introduce new syntax: the grammar is fixed in **parser.c**. Lifting that restriction — letting a reviser block teach the parser a new rule — is the subject of §3.

3. Reviser as grammar extension

3.1 Rules as committed facts

We model a production rule as a ledger fact: a tuple

```
rule := [ "rule", name, head, pattern, denotation ]
```

where **name** identifies the rule, **head** is the nonterminal it extends (**stmt**, **expr**, or **postfix**), **pattern** is a token template with holes, and **denotation** is a Bada expression — typically a call into the engine — that the matched form elaborates to. A reviser block is a *grammar transaction*: it evaluates to a sequence of **rule** tuples and commits them to a dedicated region of the ledger, the **rule ledger** $\Omega_{\text{grammar}} : \text{DATABASE}[\text{grammar}]$.

```
@reviser : grammar {
```

```

rule "hadamard" postfix [ 'H' '(' expr ')' ]
                    => phase_uniform(_1)
rule "measure"   stmt   [ 'Measure' '(' expr ')' ]
                    => commit_measurement(_1)
}

```

Two design rules keep this sound and in keeping with the engine.

Monotonicity. Rules are appended, never overwritten. A later reviser may *shadow* an earlier rule by committing a higher-priority rule with the same head, but the earlier fact remains in the ledger, so the grammar's history is auditable — the same append-only discipline the engine uses for evidence.

Scope. The rule ledger is consulted by the parser only for input that follows the committing reviser in program order (or, in module form, only within the importing scope). Syntax a program has not yet "learned" parses as before. This makes extension *local* and prevents a downloaded module from silently changing the meaning of code written before it.

3.2 The parser as a rule reader

The reference parser is recursive descent with a fixed set of productions. We extend exactly three extension points — `stmt`, `expr`'s primary, and `postfix` — each of which, before falling through to its built-in alternatives, consults the rule ledger for a rule whose `head` matches and whose `pattern` matches the upcoming tokens. On a match, the parser builds a generic `N_EXTENDED` node carrying the rule's `name` and the captured holes `_1`, `_2`, ..., and elaboration to the rule's `denotation` happens at evaluation time (interpreter) or lowering time (compiler).

This is a deliberately small surgery. The built-in grammar is unchanged; the extension points are *additive*; and because a rule's denotation is an ordinary Bada expression, no new evaluator machinery is needed beyond binding the holes. Ambiguity is resolved by rule priority (latest-highest), then by ledger order, so the resolution policy is itself a fact about the ledger rather than a hidden parser heuristic.

3.3 Why the ledger, not a macro table

A conventional extensible language keeps macros in a compile-time table. Bada keeps them in the ledger for three reasons that matter to *this* engine. First, the ledger is already the program's source of truth, so grammar and evidence live in one auditable store. Second, append-only semantics give grammar extension the same monotonic, non-destructive character the engine demands of inference — you cannot quietly delete a rule any more than you can delete a committed fact. Third, because rule denotations are engine calls, the boundary between "new syntax" and "new inference" disappears: teaching Bada the word `Measure` and teaching it to commit a measurement are the same act.

4. A Q#-style quantum sublanguage on the phase core

4.1 State

A quantum register over n qubits is a distribution-plus-phase over $V = 2^n$ basis states, represented exactly by the engine's pair `(a, theta)`: `a` holds the measurement probabilities $|\alpha_i|^2$ and `theta` holds the phases. The native phase value `psi_i = sqrt(a_i) * exp(i*theta_i)` is the amplitude vector. No new type is needed; `qubit/register` are reviser-introduced *names* for engine distributions of size a power of two.

4.2 Gates as phase commits

The reviser rules denote each gate as an operation on `(a, theta)` expressed with engine primitives.

- **H on a single qubit** maps a basis state to uniform `a = [0.5, 0.5]` and installs the appropriate relative phase via one phase module. In engine terms it is `cognitive_system(a, [[a, phi_H]], [beta_H])` with the `a` set uniform — precisely the Hadamard probability profile.
- **Phase gates** `(S, T, Rz( $\lambda$ ))` are pure phase: they add to `theta` through a phase module with the `a` left untouched. By $|\psi|^2 = a$, they change no probability — exactly the unitarity one expects of a phase gate, and a direct corollary of the engine's second theorem.
- **CNOT** is a permutation of basis indices conditioned on the control bit; on `(a, theta)` it is an index remap that the interpreter performs before the phase step, leaving the engine's numeric core to handle amplitudes.

The decisive economy is that *gates introduce no new arithmetic*. Every gate is either an index permutation (handled structurally) or a phase contribution (handled by the existing cognitive core). The engine's theorems then *are* the gate semantics.

4.3 Measurement as a ledger commit

`Measure(reg)` denotes a commit: it reads `a = |psi|^2`, records the outcome distribution as a fact in `Omega::DATABASE[tuplespace]`, and — in the inference library of §5 — uses that fact to sharpen a prior. Because confident zeros are preserved through every preceding phase step, a basis state that began with zero amplitude has zero measurement probability: the **no-resurrection** property. A gate cannot make a forbidden outcome occur, which is the quantum-mechanically correct behaviour and, here, a theorem inherited from the engine rather than a rule bolted on.

4.4 A program in the extended language

After the reviser of §3.1 has committed the quantum rules, a Bada program reads in recognisably Q#-like terms while running entirely on the phase core:

```
@reviser : grammar { ... quantum rules ... }

reg := qubit(1)          # |0>
reg := H(reg)            # uniform superposition
outcome := Measure(reg)  # commit |psi|^2 to the ledger
print("P(0), P(1) =", outcome)
```

5. A specialised quantum-inference library

The engine's reason for existing is *unknown-prior* inference, and measurement is where quantum computation meets it. We specify a small library, `Omega::Quantum`, that treats repeated measurement as Bayesian evidence over an initially unknown state.

Maximum-entropy start. Tomography of an unknown state begins from `unknown_prior(V)` — the uniform distribution, the engine's honest ignorance — not from an assumed state. Each measurement commits an outcome and *updates* the prior toward the empirical frequencies, so the estimate sharpens monotonically and its entropy decreases, exactly the engine's documented behaviour.

Append-only measurement record. Because every `Measure` is a commit, the full measurement history is the ledger trace. Re-deriving a probability estimate is a fold over committed facts; there is no mutable counter to corrupt, and two runs that produce the same estimate are guaranteed to have the same evidence, fact for fact. This is the quantum-experiment analogue of the language-level "ledger-trace equality" correctness criterion.

Zero-aware estimation. Forbidden outcomes never appear in the trace (they have probability zero and are preserved as such), so the estimator never wastes prior mass on impossible basis states. The library exposes `zeros_of` on the running estimate to report which basis states have been certified impossible — a ledger-grounded notion of a measured selection rule.

The library's surface is therefore tiny and entirely engine-native: `prepare_unknown(n)` (a 2^n maximum-entropy prior), `observe(reg)` (a measuring commit that updates the prior), `estimate()` (a fold over the trace), and `forbidden()` (`zeros_of` of the estimate). Nothing in it reaches outside the primitives the engine already provides.

6. Compiler and interpreter realisation

Carrying a reviser-extended grammar from text to native code touches three components, and in each the change is additive.

Parser. The three extension points (§3.2) gain a "consult the rule ledger" step before their built-in alternatives. A matched extended form becomes an `N_EXTENDED` node holding the rule name and captured holes. The built-in grammar is untouched, so every existing program parses identically.

Interpreter. Evaluating an `N_EXTENDED` node binds the captured holes `_1`, `_2`, `...` into a fresh scope and evaluates the rule's stored denotation — an ordinary Bada expression, typically `cognitive_system`, `update`, or a commit. Gate rules thus reduce to existing native builtins; the interpreter needs no quantum-specific arithmetic. Measurement rules reduce to

commits on the existing ledger.

Transpiler. Lowering an `N_EXTENDED` node emits the C that its denotation already lowers to. Because gates denote calls to `cognitive_system`/phase steps and measurement denotes a commit, the emitted C uses the *same* embedded runtime the Bada-to-C compiler already carries (`b_softmax`, `b_cognitive`, `b_commit`). The external native back end — `gcc -O2 ... -lm` — is reused without change. A quantum Bada program therefore compiles to a standalone native binary by the identical path an ordinary engine program does; the only new compile-time work is rule lookup, which happens before lowering and leaves no trace in the back end.

The upshot is a clean separation: reviser rules are the *only* place quantum syntax exists, the phase core is the *only* place quantum numerics exist, and the two meet through denotations that are ordinary engine calls. Adding the quantum sublanguage adds no kernel and no back end.

7. Evidence and gap analysis

We probed the central claims on the current executable. The reviser's grammar-extension front end is not yet wired into `parser.c`; to validate the *semantics* independently of that surgery, the probes invoke the engine primitives directly with the values the reviser rules denote.

7.1 A Hadamard is a phase step

Realising $H|0\rangle$ as a uniform `a = [0.5, 0.5]` with one phase module yields equal measurement probabilities and leaves them unchanged under the phase, with the measurement committed to the ledger:

```
H|0> probabilities      : 0.50000 0.50000
|psi|^2 == a (maxdiff)  : 1.11e-16
measurement committed  : 1 fact
```

The phase step is unitary to machine precision (1.11×10^{-16}), confirming §4.2's claim that pure-phase gates preserve probability by the engine's second theorem.

7.2 A forbidden transition cannot be resurrected

Preparing a register in which one basis state has zero amplitude and then applying a phase step, the zero survives:

```
amplitudes^2 : 0.62246 0.00000 0.37754
forbidden basis states (zeros) : [1]
phase preserved the zero       : true
```

The confident zero is intact after the phase step, confirming §4.3's no-resurrection property: a gate cannot make a forbidden outcome occur. This is zero-preservation, read quantum-mechanically.

7.3 What the probes do not yet show

The probes validate the *denotational* core — that gates and measurement are engine operations with the right properties — but they call the primitives directly rather than through reviser-committed syntax. Three pieces of parser work remain to make the surface syntax real:

1. **Rule-ledger lookup at the three extension points.** The parser must, before its built-in alternatives, match upcoming tokens against committed `rule` patterns. This is the one non-trivial change; it is additive and localised but it is real work.
2. **Hole capture and binding.** A matched pattern must bind `_1`, `_2`, ... for the denotation. The interpreter already has scoped environments, so binding is straightforward once the parser produces `N_EXTENDED` nodes.
3. **Priority and scope enforcement.** Latest-highest priority and program-order scoping (§3.1) must be enforced at lookup. Both are reads of ledger order, so they need no new state — only a disciplined query.

None of these requires a new numeric kernel or a new back end, which is the paper's central practical claim: the hard parts (amplitudes, unitarity, zero-preservation, native compilation) are *already done* by the engine and the existing compiler, and the remaining work is a contained, additive parser extension.

8. Related ideas

Extensible-syntax systems — Lisp macros, Racket's `#lang`, Camlp4-style preprocessors, and modern procedural macros — all let a program grow its own notation. Bada's variant is distinguished by *where* the grammar lives: in an append-only ledger shared with the program's evidence, so that syntactic and empirical growth obey one monotonic discipline. The quantum target connects to the long line of embedded quantum DSLs; what differs here is that the host's native numeric type is *already* a unit-modulus amplitude, so the embedding adds no kernel and the host's theorems supply the quantum semantics. The measurement library's "estimate by folding the commit trace" echoes event-sourced systems, specialised to maximum-entropy Bayesian updating.

9. Conclusion

Bada's `@reviser`, generalised from a value-level loop to a grammar-extension transaction that commits production rules to the ledger, turns the language into one that learns syntax the way its engine learns facts — monotonically, auditably, append-only. Targeting that mechanism at a Q#-style quantum sublanguage is the sharpest possible demonstration, because the engine's native `phase` amplitude *is* a quantum amplitude: gates become phase commits, measurement becomes a ledger commit that sharpens a maximum-entropy prior, and the engine's two theorems become unitarity of the phase layer and no-resurrection of forbidden states. The numerics and the native back end already exist and are verified — a Hadamard is unitary to 1.11×10^{-16} , a forbidden transition is preserved — so the path from "Bada" to "quantum Bada" runs entirely through a contained, additive parser extension, with the inference engine doing the hard work it was built to do.

Appendix A. The quantum-to-engine dictionary

Quantum (Q#) notion	Engine realisation		
basis-state probability <code>p_i</code>	base distribution <code>a_i = softmax(z)_i</code>		
amplitude <code>α_i = sqrt(p_i) e^{iφ_i}</code>	phase value <code>psi_i = sqrt(a_i) exp(iθeta_i)</code>		
phase <code>phi_i</code>	accumulated <code>theta_i = Sum_m beta_m H_m phi_m(i)</code>		
H (Hadamard)	uniform <code>a</code> + one phase module		
S, T, R _z (λ) (phase)	phase module, <code>a</code> untouched (unitary by `	■	<sup>2</sup> =a`)
CNOT (entangle)	conditional index permutation, then phase step		
Measure	commit `	■	<sup>2</sup> to Omega::DATABASE[tuplespace]`
forbidden transition	confident zero, preserved (no-resurrection)		
repeated measurement	append-only trace + <code>update</code> of a max-entropy prior		
selection rule (measured)	<code>zeros_of(estimate)</code>		

Appendix B. Reviser grammar-extension schema

```
@reviser : grammar {
  rule NAME HEAD PATTERN => DENOTATION
  ...
}

rule      := [ "rule", name, head, pattern, denotation ]  (a ledger fact)
head      := 'stmt' | 'expr' | 'postfix'
pattern   := [ token-or-hole ... ] ; holes are _1, _2, ...
denotation := any Bada expression in the holes (typically an engine call)

parser extension points:
```

```

primary : before built-ins, try rules with head 'expr'
postfix : after a primary, try rules with head 'postfix'
stmt    : before built-ins, try rules with head 'stmt'

```

resolution: highest priority first (latest commit wins), then ledger order;
scope: a rule applies only to input following its committing reviser.

Appendix C. Engine core equations (reference)

```

a_i      = softmax(z)_i
theta_i  = Sum_m beta_m * H_m * phi_m^i
psi_i    = sqrt(a_i) * exp( i * theta_i )      ; the native 'phase' value
q_i      = |psi_i|^2 / Sum_j |psi_j|^2 = a_i    ; |exp(i theta)| = 1

```

Theorem 1 (zero-preservation): $a_i = 0 \Rightarrow q_i = 0$.

Quantum reading: a zero-amplitude basis state cannot be resurrected.

Theorem 2 ($|\psi|^2 = a$): the phase step changes no probability.

Quantum reading: a pure-phase gate is unitary.

Verified: Hadamard $|\psi|^2 = a$ to $1.11e-16$; forbidden zero preserved.